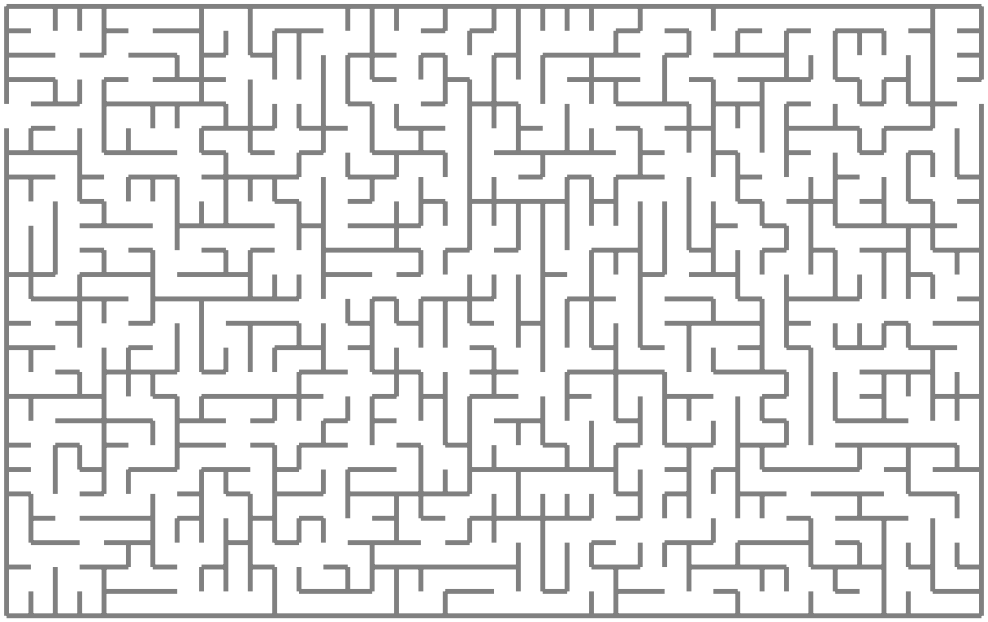
Higher order pattern unification using scope graphs

Version of February 19, 2026



Tudor Andrei

Higher order pattern unification using scope graphs

THESIS

submitted in partial fulfillment of the
requirements for the degree of

MASTER OF SCIENCE

in

COMPUTER SCIENCE

by

Tudor Andrei
born in Bucharest, Romania

Programming Languages Group
Department of Software Technology
Faculty EEMCS, Delft University of Technology
Delft, the Netherlands
www.ewi.tudelft.nl

Cover picture: Random maze.

Higher order pattern unification using scope graphs

Author: Tudor Andrei
Student id: 5495822
Email: T.Andrei@student.tudelft.nl

Abstract

This work researches the possibility of using higher order unification in a typechecker that uses scope graphs. Scope graphs are a novel data structure for name resolution created by Statix. The Statix language is part of the language workbench Spoofax, created at TuDelft, and enables users to write a typechecker for their language. Out of the box, Statix can do first order unification between terms from the user's language which is enough to typecheck the majority of type system. However, dependently typed languages present a bigger challenge, which arises from type checking definitions with implicit arguments or pattern matching. We will propose a new unification algorithm for Statix, that is able to unify higher order terms (lambda functions) while using scope graphs for name resolution.

Thesis Committee:

Chair:	Prof. dr. C. Hair, Faculty EEMCS, TU Delft
Committee Member:	Dr. A. Bee, Faculty EEMCS, TU Delft
Committee Member:	Dr. C. Dee, Faculty EEMCS, TU Delft
University Supervisor:	Ir. E. Ef, Faculty EEMCS, TU Delft

Preface

Preface here.

Tudor Andrei
Delft, the Netherlands
February 19, 2026

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Language workbenches aim to reduce the cost of designing and implementing programming languages by providing high-level, declarative abstractions for syntax, name resolution, and type checking. Spoofax, developed at TU Delft, is one such workbench. It enables users to define the syntax of a language through a grammar specification, from which an abstract syntax tree (AST) representation is automatically derived. This AST forms the basis for subsequent analyses, including name resolution and type checking.

Type checking in Spoofax is specified using Statix, a constraint-based language designed for expressing typing and name resolution rules declaratively. Statix combines term constraints with scope graphs, a graph-based representation of scopes and the relations between them. Scope graphs are particularily well suited for imports, mutual refferences and other types of non-lexical scoping.

At a conceptual level, Statix bears similarities to λ-Prolog, a higher-order logic programming language that extends Prolog with types and higher-order terms. In λ-Prolog, type systems can be encoded directly as logical specifications using predicates, quantification, and higher-order unification. Terms in λ-Prolog are written using higher order abstract syntax (HOAS) which makes the language particularly well-suited for turning formal type checker rules into an implementation.

HOAS and Scope Graphs differ fundamentally in how they treat binding and unification. λ-Prolog relies on meta-level quantification and higher-order unification to model binding, whereas Statix represents binding explicitly using scope graphs and resolves names via constraints. While scope graphs provide strong support for modular and non-lexical scoping, they do not natively provide higher-order pattern unification, which is central to many dependently typed systems.

This tension motivates the central question of this thesis: can higher-order pattern unification be integrated with a scope-graph-based constraint system in a principled and practical way? Furthermore, how does such an approach compare to higher-order logic programming systems when implementing a dependently typed language?

To address these questions, we design and evaluate a higher-order pattern unification algorithm tailored to a scope-graph-based setting, and analyze its implications for implementing dependently typed languages in Spoofax.

1.1 Contributions

This thesis addresses two research questions and makes the following contributions.

Higher-order pattern unification in scope-graph systems (RQ1). We design and formalize an algorithm for higher-order pattern unification in a constraint-based system that uses

scope graphs for name binding. To validate the algorithm, we implement a minimal dependently typed core language in Statix, demonstrating that higher-order pattern unification can be integrated with scope-graph-based name resolution and supports core dependent type features.

Comparison of implementation paradigms for dependent typing (RQ2). We provide a structured comparison between higher-order logic programming systems such as λ-Prolog and scope-graph-based constraint systems. The analysis examines differences in expressivity and computability, including the treatment of binding, fresh name generation, modularity, and non-lexical scoping. This clarifies the trade-offs between meta-level binding mechanisms and explicit scope representations when implementing practical dependently typed languages.

Chapter 2

Background: Statix

This chapter will focus on the concepts behind Statix. We also thing it is necessary to offer readers information about Spoofax and its features. It is important to understand where Statix sits in the context of the Spoofax framework to get a better insight into what's possible to do in a typechecker.

Spoofax is a framework that allows users to easily implement end-to-end tooling for their programming language. Made primarily for use within Eclipse, Spoofax comes as a plugin. In Spoofax, users start by defining the grammar of the language using SDF3. Later, one can write various pases, such as desugaring steps, IDE services or an interpreter, on the AST with Stratego and typing rules using Statix.

2.1 Parsing the AST

Getting an AST representation of the source code is one of the first things needed to compile or typecheck a language. This process gives meaning to the tokens found in a source code file. In SDF3, the parsing rules are defined declaratively, similarly to BNF notation. You can define context-free sorts with multiple constructors. The constructors can match on code tokens or other sorts. As an example, users might find it helpful to define a sort for expressions and another for types. The parsing starts from a top level sort and will try to match the tokens with the defined constructors. As an example consider the grammar in figure.

```
context-free sorts
  Expr
  Type

context-free syntax
  Expr.Num = Number
  Expr.Eq = [[Expr] == [Expr]] {left}
  Expr.Add = [[Expr] + [Expr]] {left}
  Expr.Var = Name
  Expr.Lam = [\[Name] : [Type] -> [Expr]]
  Expr.App = [[Expr] [Expr]] {right}

  Type.Nat = [nat]
  Type.Fun = [[Type] -> [Type]]
context-free priorities
  {Expr.Num Expr.Var} > Expr.App > Expr.Add > Expr.Eq > Expr.Lam
```

Listing 2.1: Simple language with Type and Expr sorts

The AST is in **ATerm** format and will contain the constructors that were matched. For instance, the program `\x : Int -> x + 1` , will produce the AST:

```
Expr.Lam("x", Type.Nat(), Expr.Add(Expr.Var("x"), Number(1)))
```

2.2 Unification

This section will explain what unification is, where it is used and the different types of algorithms.

In general unification is concerned with filling in missing parts of terms in order to make them equal. Unification wants to know if two terms A and B are equal, and if not, if they can be made equal by filling in their free variables. For example, consider the unification problem $f(x, y) \equiv f(1, g(2))$ (assume f and g are known functions). The algorithm should find the substitutions $x \mapsto 1, y \mapsto g(2)$.

Common requirements for a unification algorithm are decidability and the generality of the solution. Decidability is a natural requirement, so it is interesting to discuss what generality of the solution means. We would prefer if our algorithm will find the substitution that doesn't constraint the term more than it needs to. Consider the problem of unifying the terms $f(x, y) \equiv f(y, x)$. A possible solution is the substitution $x \mapsto 1, y \mapsto 1$. However this solution constraints the free variables too much. The most general solution to this problem is $x \mapsto y$. Given the difficulty of unification, one may wonder if a decidable algorithm exists for the most general unifier (MGU). To answer this, we must make a distinction on the type of terms we will substitute for the free variables. The biggest distinction is between so called first order and higher order terms. First order terms are variables, constants and in general terms that dont bind new variables. First order unification is decidable and admits the most general unifier.

2.3 Statix

Statix is used by Spoofax uses to perform typechecking of the user's language. Statix is based on terms and unification. The main concepts of Statix are:

- **Terms**: This is the smallest abstraction that Statix provides. Statix terms can be unified using the first order unification algorithm built into the language.
- **Rules**: These are similar to predicates in Prolog. Given an instantiation of a rule, Statix will try to prove it. Rules can pattern match on their arguments, and in each case will require some constraint(s) to hold in order for them to be proven.
- **Constraints**: Constraints are problems that Statix will work on involving term and scope graph operations. Possible constraints are (dis)equality, conjunction, disjunction and scope graph constraints. For example, a rule can require that two terms are equal, or that a new scope is added to the scope graph.

You can introduce variables in the rule head, but at runtime it will receive an actual value. Variables introduced as existential will be filled by unification

Terms in Statix follow a sorted logic. The user can declare new sorts and constructors for objects in the sort. This design differs from λ-Prolog as you cannot create lambda terms. They are not first class citizens in Statix. Moreover, Statix does not perform function application on terms.

Rules are the Statix equivalent of predicates in λ-Prolog. In λ-Prolog, the convention is that the output of a predicate is stored in its last argument. The caller will provide a fresh, unfilled variable to the predicate. When the predicate is proven, the variable will be filled with a value through unification. This is also the case in Statix. However, Statix also

offers functional rules, which mimic functions in functional programming languages. Functional rules are converted by Statix to regular rules as syntactic sugar. When simplyfying constraints, Statix doesn't backtrack. This is different to λProlog implementations such as Elpi(()). Statix will try at most one rule for each constraint. The appropriate rule is selected by applying the following heuristics in order: 1. Rules with a smaller domain are preferred over rules with a larger domain. 2. When pairwise comparing rules, the rule for which, in left-to-right order, a more specific pattern is encountered first is preferred over the other. For all the cases where the heuristic can't decide, Statix emits compile time errors.

Although the Statix language does not distinguish between variables in rule heads and existentials, the solver treats those quite differently. On rule application, occurrences of variables from rule heads in the body of the rule are substituted by the actual arguments. For example, in a specification containing the rule rule(x) :- x == ()., simplifying the constraint rule(()) will result in the residual constraint () == (), which trivially holds. However, when simplifying the constraint x x == (), simplifying will generate a fresh unification variable (say ?x-1), and substitute that in the constraint, yielding ?x-1 == (). Solving that will create a new mapping ?x-1 |-> () in the unifier of the solver result.

2.4 Scope Graphs

Chapter 3

Evaluation

Evaluation here.

Chapter 4

Related work

Related work here.

Chapter 5

Conclusion

Conclusion here.

Acronyms

AST abstract syntax tree

DSL domain-specific language

Appendix A

A

Appendix here.